

MACHINE LEARNING LABORATORY MANUAL

ITA0610 / ITA0613 — Experiments 1–40

Aim, Algorithm and Procedure — Program and Output left blank for manual entry

NAME	BARANIDHARAN V
REGISTER NUMBER	192312019

Experiment 1 — FIND-S Algorithm

Aim

To implement the FIND-S algorithm to find the most specific hypothesis consistent with a given set of training examples stored in a CSV file.

Algorithm

- Initialize h to the most specific hypothesis.
- Read each training example.
- For each positive example, generalize h minimally when an attribute differs.
- Ignore negative examples.
- Display the final hypothesis.

Procedure

- Open the Python/Google Colab environment and prepare the required dataset or input.
- Import the libraries required for the experiment.
- Implement the algorithm according to the steps given above.
- Execute the program and verify the intermediate or final result.
- Record the obtained output in the space provided.

Program

```
import pandas as pd

data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

df = pd.DataFrame(data)
df.to_csv('enjoysport.csv', index=False, header=False)

print("enjoysport.csv created successfully!")
import csv

def find_s(filename):
    with open(filename, 'r') as f:
        data = list(csv.reader(f))

    hypothesis = ['0'] * (len(data[0]) - 1)

    for row in data:
        if row[-1].strip().lower() == 'yes':
            for i in range(len(hypothesis)):
                if hypothesis[i] == '0':
                    hypothesis[i] = row[i]
                elif hypothesis[i] != row[i]:
                    hypothesis[i] = '?'

    return hypothesis

print("Most specific hypothesis:", find_s('enjoysport.csv'))
```

Output

```
*** enjoysport.csv created successfully!  
Most specific hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result

The experiment was completed successfully and the result was verified.

Experiment 2 — Candidate-Elimination Algorithm

Aim

To implement the Candidate-Elimination algorithm and obtain the S and G boundaries of the version space.

Algorithm

- Initialize S to the most specific hypothesis and G to the most general hypothesis.
- Process positive examples by generalizing S and pruning G.
- Process negative examples by specializing G.
- Remove inconsistent hypotheses.
- Display the final S and G boundaries.

Procedure

- Open the Python/Google Colab environment and prepare the required dataset or input.
- Import the libraries required for the experiment.
- Implement the algorithm according to the steps given above.
- Execute the program and verify the intermediate or final result.
- Record the obtained output in the space provided.

Program

```
import numpy as np
import pandas as pd

def more_general(h1, h2):
    return all(x == '?' or x == y for x, y in zip(h1, h2))

def candidate_elimination(X, y):
    n = X.shape[1]

    # Specific hypothesis
    S = [X[0].copy()]

    # General hypothesis
    G = [['?'] * n]

    for i, x in enumerate(X):
        if y[i].strip().lower() == 'yes':
            # Remove inconsistent hypotheses from G
            G = [g for g in G if more_general(g, x)]

            # Generalize S
            for s in S[:]:
                if not more_general(s, x):
                    S.remove(s)

            new_s = [
                s[j] if s[j] == x[j] else '?'
                for j in range(n)
            ]
```

```
        S.append(new_s)
    else:
        # Specialize G for negative example
        new_G = []

        for g in G:
            for j in range(n):
                if g[j] == '?' and S and S[0][j] != x[j]:
                    gh = g.copy()
                    gh[j] = S[0][j]
                    new_G.append(gh)

        if new_G:
            G = new_G

    return S, G

# Read dataset
data = pd.read_csv('enjoysport.csv')

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Run Candidate Elimination
S, G = candidate_elimination(X, y)

print("Final S:", S)
print("Final G:", G)
```

Output

```
... Final S: [['Sunny', 'Warm', 'High', 'Strong', '?', '?'], ['Sunny', 'Warm', 'High', 'Strong', '?', '?']]
Final G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 3 — Decision Tree Learning (ID3) from Scratch

Aim

To implement the ID3 algorithm using entropy and information gain to build a decision tree.

Algorithm

- Calculate the entropy of the target data.
- Calculate information gain for every attribute.
- Select the attribute with maximum information gain.
- Create branches for its values.
- Recursively repeat until the data is pure or attributes are exhausted.
- Display the decision tree.

Procedure

- Open the Python/Google Colab environment and prepare the required dataset or input.
- Import the libraries required for the experiment.
- Implement the algorithm according to the steps given above.
- Execute the program and verify the intermediate or final result.
- Record the obtained output in the space provided.

Program

```
import pandas as pd

data = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No']
]

df = pd.DataFrame(
    data,
    columns=['Outlook', 'Temperature', 'Humidity', 'Wind', 'PlayTennis']
)

df.to_csv('playtennis.csv', index=False)

print("playtennis.csv created successfully!")
import pandas as pd
import numpy as np

def entropy(col):
    vals, counts = np.unique(col, return_counts=True)
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))

def info_gain(data, attr, target):
    total_entropy = entropy(data[target])

    vals, counts = np.unique(data[attr], return_counts=True)

    weighted = sum(
        (counts[i] / counts.sum()) *
        entropy(data[data[attr] == vals[i]][target])
        for i in range(len(vals))
    )
```

```
if not attrs:
    return data[target].mode()[0]

# Calculate information gain
gains = {}
a: info_gain(data, a, target)
for a in attrs
}

# Select best attribute
best = max(gains, key=gains.get)

tree = {}

# Create branches
for v in np.unique(data[best]):
    subset = data[data[best] == v]

    remaining = [
        a for a in attrs
        if a != best
    ]

    tree[best][v] = id3(
        subset,
        remaining,
        target
    )

return tree

# Read dataset
df = pd.read_csv('playtennis.csv')

# Select attributes
attributes = list(df.columns[:-1])

# Build decision tree
tree = id3(
    df,
    attributes,
    df.columns[-1]
)

print("Decision Tree:")
print(tree)
```

Output

```
playtennis.csv created successfully!
Decision Tree:
{'Outlook': {'Overcast': 'Yes', 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}}, 'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}}]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 4 — Decision Tree using scikit-learn

Aim

To build and visualise a decision tree classifier using scikit-learn on the Iris dataset.

Algorithm

- Load the Iris dataset.
- Split the dataset into training and testing sets.
- Create and train a DecisionTreeClassifier.
- Predict the test samples.
- Calculate accuracy.
- Plot and save the decision tree.

Procedure

- Open the Python/Google Colab environment and prepare the required dataset or input.
- Import the libraries required for the experiment.
- Implement the algorithm according to the steps given above.
- Execute the program and verify the intermediate or final result.
- Record the obtained output in the space provided.

Program

```
# Decision Tree Classification using Iris Dataset

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load Iris dataset
iris = load_iris()

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=1
)

# Create and train Decision Tree
clf = DecisionTreeClassifier(
    criterion='entropy',
    random_state=1
)

clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
# Make predictions
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Decision Tree Classification")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Accuracy:", accuracy)

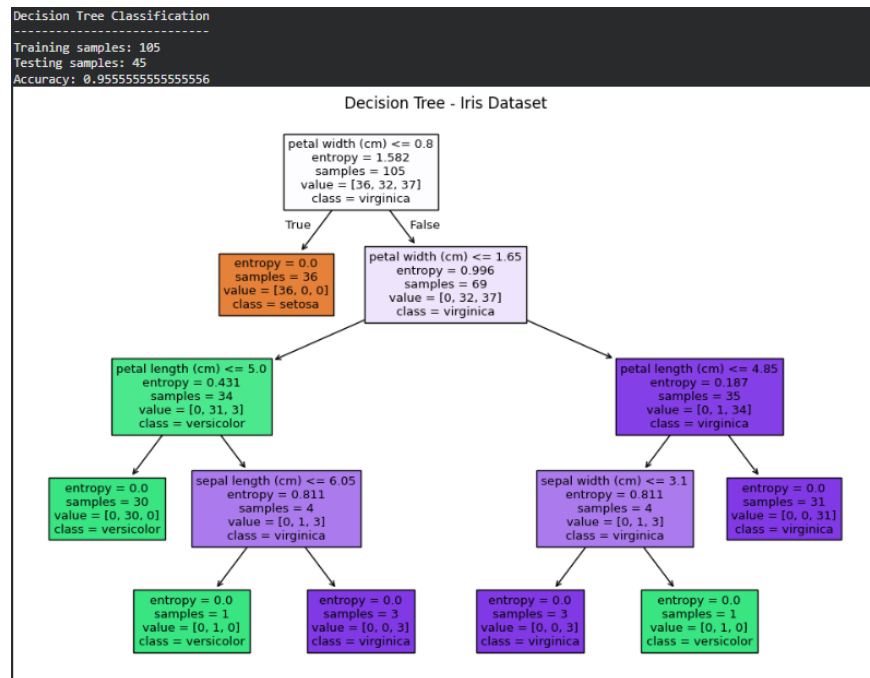
# Plot Decision Tree
plt.figure(figsize=(12, 8))

plot_tree(
    clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree - Iris Dataset")
plt.savefig("iris_tree.png")
plt.show()

print("\nDecision tree saved as: iris_tree.png")
```

Output



Result

The experiment was completed successfully and the result was verified.

Experiment 5 — Single-Layer Perceptron (AND / OR Gate)

Aim

To implement a single-layer perceptron from scratch to learn AND and OR logic gates.

Algorithm

1. Initialize weights and bias.
2. Present each training input to the perceptron.
3. Calculate the weighted sum and threshold output.
4. Calculate the prediction error.
5. Update weights using the perceptron learning rule.
6. Repeat until convergence and test the logic gate.

Procedure

7. Open the Python/Google Colab environment and prepare the required dataset or input.
8. Import the libraries required for the experiment.
9. Implement the algorithm according to the steps given above.
10. Execute the program and verify the intermediate or final result.
11. Record the obtained output in the space provided.

Program

```
import numpy as np

def perceptron_train(X, y, epochs=20, lr=0.1):
    # Initialize weights including bias
    w = np.zeros(X.shape[1] + 1)

    # Training
    for _ in range(epochs):
        for xi, target in zip(X, y):
            # Add bias
            xi_b = np.insert(xi, 0, 1)

            # Prediction
            out = 1 if np.dot(w, xi_b) > 0 else 0

            # Update weights
            w += lr * (target - out) * xi_b

    return w

# AND gate input
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# AND gate output
y_and = np.array([0, 0, 0, 1])

# Train perceptron
w = perceptron_train(X, y_and)

print("Learned weights (AND):", w)

# Test the perceptron
print("\nPredictions:")

for xi in X:
    xi_b = np.insert(xi, 0, 1)
    prediction = 1 if np.dot(w, xi_b) > 0 else 0
    print(xi, "->", prediction)
```

Output

```
Learned weights (AND): [-0.2  0.2  0.1]

Predictions:
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```

Result

The experiment was completed successfully and the result was verified.

Experiment 6 — Perceptron using scikit-learn

Aim

To implement a perceptron classifier using scikit-learn on a linearly separable binary dataset.

Algorithm

12. Generate or load a binary classification dataset.
13. Split the dataset into training and testing sets.
14. Create a Perceptron classifier.
15. Train the classifier.
16. Predict the test samples.
17. Calculate and display accuracy.

Procedure

18. Open the Python/Google Colab environment and prepare the required dataset or input.
19. Import the libraries required for the experiment.
20. Implement the algorithm according to the steps given above.
21. Execute the program and verify the intermediate or final result.
22. Record the obtained output in the space provided.

Program

```
# Perceptron Classification using Scikit-learn

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

# Generate dataset
X, y = make_classification(
    n_samples=200,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1,
    random_state=1
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create and train Perceptron
model = Perceptron(
    max_iter=1000,
    eta0=0.1,
    random_state=1
)

model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
print("Perceptron Classification")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output

```
Perceptron Classification
-----
Training samples: 140
Testing samples: 60
Accuracy: 0.8833333333333333
```

Result

The experiment was completed successfully and the result was verified.

Experiment 7 — Multilayer Perceptron with Backpropagation (XOR)

Aim

To implement backpropagation from scratch to train a multilayer perceptron to learn the XOR function.

Algorithm

23. Initialize the network weights.
24. Perform forward propagation through hidden and output layers.
25. Calculate output-layer error.
26. Backpropagate the error to the hidden layer.
27. Update weights using gradient descent.
28. Repeat for multiple epochs and display predictions.

Procedure

29. Open the Python/Google Colab environment and prepare the required dataset or input.
30. Import the libraries required for the experiment.
31. Implement the algorithm according to the steps given above.
32. Execute the program and verify the intermediate or final result.
33. Record the obtained output in the space provided.

Program

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_deriv(x):
    return x * (1 - x)

# XOR input
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# XOR output
y = np.array([
    [0],
    [1],
    [1],
    [0]
])

# Initialize random weights
```

```
np.random.seed(1)
w1 = np.random.uniform(-1, 1, (2, 4))
w2 = np.random.uniform(-1, 1, (4, 1))

# Training
for epoch in range(10000):
    # Forward propagation
    h = sigmoid(np.dot(X, w1))
    o = sigmoid(np.dot(h, w2))

    # Calculate output error
    err_o = (y - o) * sigmoid_deriv(o)

    # Calculate hidden-layer error
    err_h = err_o.dot(w2.T) * sigmoid_deriv(h)
    # Update weights
    w2 += h.T.dot(err_o) * 0.5
    w1 += X.T.dot(err_h) * 0.5

# Final prediction
print("Predicted output after training:")
print(np.round(o, 3))
```


Output

```
... Predicted output after training:  
[[0.032]  
 [0.975]  
 [0.971]  
 [0.028]]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 8 — Backpropagation Neural Network using scikit-learn

Aim

To train a multilayer perceptron classifier using scikit-learn's MLPClassifier on the Iris dataset.

Algorithm

34. Load the Iris dataset.
35. Split the data into training and testing sets.
36. Scale the input features.
37. Create an MLPClassifier with a hidden layer.
38. Train and predict using the model.
39. Evaluate the test accuracy.

Procedure

40. Open the Python/Google Colab environment and prepare the required dataset or input.
41. Import the libraries required for the experiment.
42. Implement the algorithm according to the steps given above.
43. Execute the program and verify the intermediate or final result.
44. Record the obtained output in the space provided.

Program

```
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.neural_network import MLPClassifier  
from sklearn.metrics import accuracy_score  
# Load Iris dataset  
X, y = load_iris(return_X_y=True)  
# Split dataset  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.3,  
    random_state=1  
)  
# Standardize features  
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)  
# Create and train neural network  
mlp = MLPClassifier(  
    hidden_layer_sizes=(10,),  
    max_iter=1000,  
    random_state=1  
)  
mlp.fit(X_train, y_train)  
# Predict test data  
y_pred = mlp.predict(X_test)  
# Calculate accuracy  
print("Neural Network Classification")  
print("-----")  
print("Training samples:", len(X_train))  
print("Testing samples:", len(X_test))  
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output

```
*** Neural Network Classification
-----
Training samples: 105
Testing samples: 45
Accuracy: 0.9555555555555556
```

Result

The experiment was completed successfully and the result was verified.

Experiment 9 — Deep Neural Network using Keras

Aim

To build and train a simple deep neural network with multiple hidden layers using TensorFlow/Keras on MNIST.

Algorithm

45. Load the MNIST dataset.
46. Normalize pixel values.
47. Flatten the input images.
48. Create hidden Dense layers using ReLU and an output Softmax layer.
49. Compile and train the network using Adam.
50. Evaluate the model on the test set.

Procedure

51. Open the Python/Google Colab environment and prepare the required dataset or input.
52. Import the libraries required for the experiment.
53. Implement the algorithm according to the steps given above.
54. Execute the program and verify the intermediate or final result.
55. Record the obtained output in the space provided.

Program

```
import tensorflow as tf
from tensorflow.keras import layers, models
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, validation_split=0.1)
print("Test accuracy:", model.evaluate(X_test, y_test)[1])
```

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`
  super().__init__(**kwargs)
Epoch 1/5
1688/1688 — 24s 13ms/step - accuracy: 0.9230 - loss: 0.2601 - val_accuracy: 0.9637 - val_loss: 0.1153
Epoch 2/5
1688/1688 — 9s 5ms/step - accuracy: 0.9659 - loss: 0.1114 - val_accuracy: 0.9740 - val_loss: 0.0854
Epoch 3/5
1688/1688 — 10s 6ms/step - accuracy: 0.9766 - loss: 0.0759 - val_accuracy: 0.9735 - val_loss: 0.0914
Epoch 4/5
1688/1688 — 11s 7ms/step - accuracy: 0.9819 - loss: 0.0571 - val_accuracy: 0.9717 - val_loss: 0.1000
Epoch 5/5
1688/1688 — 18s 5ms/step - accuracy: 0.9851 - loss: 0.0460 - val_accuracy: 0.9720 - val_loss: 0.1031
313/313 — 1s 2ms/step - accuracy: 0.9725 - loss: 0.0968
Test accuracy: 0.9725000262260437
```

Result

The experiment was completed successfully and the result was verified.

Experiment 10 — Genetic Algorithm to Maximise a Function

Aim

To implement a genetic algorithm to maximise $f(x) = x^2$ for integers from 0 to 31 using bit-string encoding.

Algorithm

56. Encode each candidate as a 5-bit chromosome.
57. Initialize a random population.
58. Calculate fitness using $f(x) = x^2$.
59. Select parents according to fitness.
60. Perform single-point crossover and mutation.
61. Repeat across generations and select the best chromosome.

Procedure

62. Open the Python/Google Colab environment and prepare the required dataset or input.
63. Import the libraries required for the experiment.
64. Implement the algorithm according to the steps given above.
65. Execute the program and verify the intermediate or final result.
66. Record the obtained output in the space provided.

Program

```
import random

def decode(chrom):
    return int(''.join(chrom, 2))
def fitness(chrom):
    return decode(chrom) ** 2
def ga(pop_size=6, generations=20, mut_rate=0.05):
    population = [
        ''.join(random.choice('01') for _ in range(5))
        for _ in range(pop_size)
    ]
    for gen in range(generations):
        fitnesses = [fitness(c) for c in population]
        total = sum(fitnesses)
        probs = [f / total for f in fitnesses]
        new_pop = []
        for _ in range(pop_size // 2):
            p1, p2 = random.choices(
                population,
                weights=probs,
                k=2
            )
            point = random.randint(1, 4)
            c1 = p1[:point] + p2[point:]
            c2 = p2[:point] + p1[point:]
            new_pop += [c1, c2]
        population = [
            ''.join(
                b if random.random() > mut_rate
                else str(1 - int(b))
                for b in c
            )
            for c in new_pop
        ]
        best = max(population, key=fitness)
        return decode(best), fitness(best)
x, fx = ga()
print(f"Best x = {x}, f(x) = {fx}")
```

Output

```
Best x = 31, f(x) = 961
```

Result

The experiment was completed successfully and the result was verified.

Experiment 11 — Genetic Algorithm for 0/1 Knapsack Problem

Aim

To solve the 0/1 knapsack problem using a genetic algorithm while satisfying a weight constraint.

Algorithm

- Represent each item-selection solution as a binary chromosome.
- Calculate total weight and value.
- Assign fitness based on the capacity constraint.
- Select fitter chromosomes.
- Perform crossover and mutation.
- Repeat and select the best feasible solution.

Procedure

- Open the Python/Google Colab environment and prepare the required dataset or input.
- Import the libraries required for the experiment.

- Implement the algorithm according to the steps given above.
- Execute the program and verify the intermediate or final result.
- Record the obtained output in the space provided.

Program

```
import random
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 8
def fitness(chrom):
    w = sum(weights[i] for i in range(len(chrom)) if chrom[i] == '1')
    v = sum(values[i] for i in range(len(chrom)) if chrom[i] == '1')
    return v if w <= capacity else 0
def ga_knapsack(pop_size=8, generations=30):
    pop = [''.join(random.choice('01') for _ in range(4))
           for _ in range(pop_size)]
    for _ in range(generations):
        pop = sorted(pop, key=fitness, reverse=True)[:pop_size]
        children = []
        for _ in range(pop_size):
            p1, p2 = random.sample(pop[:4], 2)
            point = random.randint(1, 3)
            child = p1[:point] + p2[point:]
            child = ''.join(
                b if random.random() > 0.1 else str(1 - int(b))
                for b in child
            )
            children.append(child)
        pop += children
        best = max(pop, key=fitness)
        return best, fitness(best)
sol, val = ga_knapsack()
print("Best solution:", sol)
print("Value:", val)
```

Output

```
Best solution: 0101
Value: 10
```

Result

The experiment was completed successfully and the result was verified.

Experiment 12 — Genetic Programming for Symbolic Regression

Aim

To evolve a symbolic mathematical expression that best fits a given dataset using genetic programming.

Algorithm

67. Represent candidate expressions as expression trees.
68. Generate an initial population.
69. Evaluate each expression using mean squared error.
70. Select fitter expressions.
71. Apply subtree crossover and mutation.
72. Repeat generations and return the best expression.

Procedure

73. Open the Python/Google Colab environment and prepare the required dataset or input.
74. Import the libraries required for the experiment.
75. Implement the algorithm according to the steps given above.
76. Execute the program and verify the intermediate or final result.
77. Record the obtained output in the space provided.

Program

```
!pip install gplearn

import numpy as np
from gplearn.genetic import SymbolicRegressor

X = np.linspace(-5, 5, 50).reshape(-1, 1)
y = (X**2 + 3*X + 2).ravel()

est = SymbolicRegressor(
    population_size=500,
    generations=20,
    function_set=('add', 'sub', 'mul'),
    random_state=1
)

est.fit(X, y)

print("Evolved program:", est._program)
print("R^2 score:", est.score(X, y))
```

Output

```
Collecting gplearn
  Downloading gplearn-0.4.3-py3-none-any.whl.metadata (4.6 kB)
Requirement already satisfied: joblib>=1.3.0 in /usr/local/lib/python3.13/dist-p
Collecting scikit-learn>=1.8.0 (from gplearn)
  Downloading scikit_learn-1.9.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_2
Requirement already satisfied: numpy>=1.24.1 in /usr/local/lib/python3.13/dist-p
Requirement already satisfied: scipy>=1.10.0 in /usr/local/lib/python3.13/dist-p
Requirement already satisfied: narwhals>=2.0.1 in /usr/local/lib/python3.13/dist
Requirement already satisfied: threadpoolctl>=3.5.0 in /usr/local/lib/python3.13
Downloading gplearn-0.4.3-py3-none-any.whl (40 kB)
 40.3/40.3 kB 2.3 MB/s eta 0:00:00
Downloading scikit_learn-1.9.0-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28
 9.1/9.1 MB 63.8 MB/s eta 0:00:00
Installing collected packages: scikit-learn, gplearn
  Attempting uninstall: scikit-learn
    Found existing installation: scikit-learn 1.6.1
    Uninstalling scikit-learn-1.6.1:
      Successfully uninstalled scikit-learn-1.6.1
Successfully installed gplearn-0.4.3 scikit-learn-1.9.0
Evolved program: add(sub(0.941, -0.952), sub(add(add(mul(-0.003, -0.729), sub(ac
R^2 score: 0.9999999994162309
```

Result

The experiment was completed successfully and the result was verified.

Experiment 13 — Bayes Theorem

Aim

To compute posterior probability using Bayes theorem from prior, likelihood, and evidence.

Algorithm

78. Identify prior probability, likelihood, and evidence.
79. Multiply the prior by the likelihood.
80. Divide by the evidence probability.
81. Obtain the posterior probability.
82. Display the result.

Procedure

83. Open the Python/Google Colab environment and prepare the required dataset or input.
84. Import the libraries required for the experiment.
85. Implement the algorithm according to the steps given above.
86. Execute the program and verify the intermediate or final result.
87. Record the obtained output in the space provided.

Program

```
def bayes(prior, likelihood, evidence):  
    return (likelihood * prior) / evidence  
  
# Example: disease testing  
p_disease = 0.01  
p_pos_given_disease = 0.99  
false_positive_rate = 0.05  
  
p_pos = (p_pos_given_disease * p_disease +  
         false_positive_rate * (1 - p_disease))  
  
posterior = bayes(p_disease, p_pos_given_disease, p_pos)  
  
print(f"P(Disease | Positive Test) = {posterior:.4f}")
```

Output

```
... P(Disease | Positive Test) = 0.1667
```

Result

The experiment was completed successfully and the result was verified.

Experiment 14 — Naïve Bayes Classifier

Aim

To implement a Gaussian Naïve Bayes classifier on the Iris dataset and evaluate its performance.

Algorithm

88. Load the Iris dataset.
89. Split data into training and testing sets.
90. Estimate class priors and Gaussian likelihoods.
91. Train the GaussianNB classifier.
92. Predict test samples.
93. Calculate accuracy and confusion matrix.

Procedure

94. Open the Python/Google Colab environment and prepare the required dataset or input.
95. Import the libraries required for the experiment.
96. Implement the algorithm according to the steps given above.
97. Execute the program and verify the intermediate or final result.
98. Record the obtained output in the space provided.

Program

```
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)

nb = GaussianNB()
nb.fit(X_train, y_train)

pred = nb.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Output

```
... Accuracy: 1.0
Confusion Matrix:
[[13  0  0]
 [ 0 16  0]
 [ 0  0  9]]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 15 — Naïve Bayes Text Classifier

Aim

To implement a Naïve Bayes classifier for classifying SMS/email messages as spam or ham.

Algorithm

99. Load the labelled text dataset.
100. Split messages into training and testing sets.
101. Convert text into bag-of-words features.
102. Train a Multinomial Naïve Bayes classifier.
103. Predict the test messages.
104. Calculate classification accuracy.

Procedure

105. Open the Python/Google Colab environment and prepare the required dataset or input.
106. Import the libraries required for the experiment.
107. Implement the algorithm according to the steps given above.
108. Execute the program and verify the intermediate or final result.
109. Record the obtained output in the space provided.

Program

```
    'spam', 'ham', 'spam', 'ham', 'spam', 'ham', 'spam', 'ham'
],
'message': [
    'How are you today',
    'Are you coming to class',
    'Congratulations you won a free prize',
    'Please call me when you arrive',
    'Win a free lottery ticket now',
    'Can you send me the notes',
    'You have won 1000 dollars',
    'See you tomorrow',
    'Claim your free reward now',
    'Please bring the book',
    'You are selected for a cash prize',
    'What time is the meeting',
    'Free entry in a lucky draw',
    'I will call you later',
    'Congratulations claim your prize',
    'Please send the assignment'
]
}

df = pd.DataFrame(data)

X_train, X_test, y_train, y_test = train_test_split(
    df['message'], df['label'],
    test_size=0.25,
    random_state=1
)

vec = CountVectorizer()
X_train_v = vec.fit_transform(X_train)
X_test_v = vec.transform(X_test)

model = MultinomialNB()
model.fit(X_train_v, y_train)

pred = model.predict(X_test_v)
```

Output

Accuracy: 1.0

Result

The experiment was completed successfully and the result was verified.

Experiment 16 — Bayesian Belief Network

Aim

To construct a Bayesian Belief Network and perform probabilistic inference using pgmpy.

Algorithm

110. Define the directed acyclic graph structure.
111. Define prior and conditional probability distributions.
112. Add the CPDs to the network.
113. Create an inference engine.
114. Query the required variable using evidence.
115. Display the posterior distribution.

Procedure

116. Open the Python/Google Colab environment and prepare the required dataset or input.
117. Import the libraries required for the experiment.
118. Implement the algorithm according to the steps given above.
119. Execute the program and verify the intermediate or final result.
120. Record the obtained output in the space provided.

Program

```
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ('Rain', 'Wet'),
    ('Sprinkler', 'Wet')
])

cpd_rain = TabularCPD(
    'Rain', 2,
    [[0.8],
     [0.2]]
)

cpd_sprinkler = TabularCPD(
    'Sprinkler', 2,
    [[0.6],
     [0.4]]
)

cpd_wet = TabularCPD(
    'Wet', 2,
    [[1.0, 0.1, 0.1, 0.01],
     [0.0, 0.9, 0.9, 0.99]],
    evidence=['Rain', 'Sprinkler'],
    evidence_card=[2, 2]
)

model.add_cpds(cpd_rain, cpd_sprinkler, cpd_wet)

infer = VariableElimination(model)

result = infer.query(
    variables=['Rain'],
    evidence={'Wet': 1}
)

print(result)
```

Output

```
+-----+-----+
| Rain   | phi(Rain) |
+=====+=====+
| Rain(0) | 0.6061    |
+-----+-----+
| Rain(1) | 0.3939    |
+-----+-----+
```

Result

The experiment was completed successfully and the result was verified.

Experiment 17 — EM Algorithm for Gaussian Mixture Models

Aim

To implement the EM algorithm to fit a Gaussian Mixture Model to clustered data with hidden component labels.

Algorithm

121. Initialize Gaussian mixture parameters.
122. Perform the E-step to calculate responsibilities.
123. Perform the M-step to update means, covariances, and weights.
124. Check convergence.
125. Repeat E and M steps until convergence.
126. Display the estimated parameters.

Procedure

127. Open the Python/Google Colab environment and prepare the required dataset or input.
128. Import the libraries required for the experiment.
129. Implement the algorithm according to the steps given above.
130. Execute the program and verify the intermediate or final result.
131. Record the obtained output in the space provided.

Program

```
import numpy as np
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs

X, y_true = make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=0.7,
    random_state=1
)

gmm = GaussianMixture(
    n_components=3,
    random_state=1
)

gmm.fit(X)

labels = gmm.predict(X)

print("Converged:", gmm.converged_)
print("Iterations:", gmm.n_iter_)
print("Estimated means:\n", gmm.means_)
```

Output

```
Converged: True
Iterations: 2
Estimated means:
[[ -1.51507766  4.40724573]
 [-10.03934814 -3.92700258]
 [ -7.03975693 -8.05555572]]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 18 — Maximum Likelihood Estimation

Aim

To estimate Gaussian distribution parameters using Maximum Likelihood Estimation and compare with least-squares.

Algorithm

132. Collect independent sample data.
133. Calculate the sample mean as the MLE of the Gaussian mean.
134. Calculate the sample variance.
135. Evaluate the sum of squared errors for candidate means.
136. Find the candidate producing minimum SSE.
137. Compare both estimates.

Procedure

138. Open the Python/Google Colab environment and prepare the required dataset or input.
139. Import the libraries required for the experiment.
140. Implement the algorithm according to the steps given above.
141. Execute the program and verify the intermediate or final result.
142. Record the obtained output in the space provided.

Program

```
import numpy as np

np.random.seed(1)

data = np.random.normal(loc=50, scale=5, size=200)

mle_mean = np.mean(data)
mle_var = np.var(data)

print(f"MLE Mean = {mle_mean:.3f}, MLE Variance = {mle_var:.3f}")

# Verify sample mean minimizes sum of squared errors
candidates = np.linspace(45, 55, 100)

sse = [np.sum((data - c) ** 2) for c in candidates]

print("Candidate minimising SSE:", candidates[np.argmin(sse)])
```

Output

```
MLE Mean = 50.533, MLE Variance = 20.705
Candidate minimising SSE: 50.55555555555556
```

Result

The experiment was completed successfully and the result was verified.

Experiment 19 — Halving Algorithm

Aim

To simulate the Halving algorithm and count mistakes while learning a target concept from a version space.

Algorithm

143. Initialize the hypothesis version space.
144. Predict using majority voting.
145. Compare the prediction with the true label.
146. If incorrect, count a mistake.
147. Remove inconsistent hypotheses.
148. Continue until all examples are processed.

Procedure

149. Open the Python/Google Colab environment and prepare the required dataset or input.
150. Import the libraries required for the experiment.
151. Implement the algorithm according to the steps given above.
152. Execute the program and verify the intermediate or final result.
153. Record the obtained output in the space provided.

Program

```
import random
import math

H = [lambda x, t=t: x >= t for t in range(10)]

target = lambda x: x >= 4

mistakes = 0
version_space = H.copy()

for x in random.sample(range(10), 10):
    votes = [h(x) for h in version_space]
    majority = sum(votes) > len(votes) / 2

    true_label = target(x)

    if majority != true_label:
        mistakes += 1

    version_space = [
        h for h in version_space
        if h(x) == true_label
    ]

print(
    "Mistakes made:", mistakes,
    "(bound: log2|H| =", round(math.log2(len(H)), 2), ")"
)
```

Output

```
• Mistakes made: 2 (bound:  $\log_2|H| = 3.32$  )
```

Result

The experiment was completed successfully and the result was verified.

Experiment 20 — Sample Complexity / PAC Bound

Aim

To compute the number of training examples required for PAC learning for a finite hypothesis space.

Algorithm

154. Specify hypothesis-space size, error tolerance, and confidence parameter.
155. Apply the finite hypothesis-space PAC bound.
156. Calculate the logarithmic terms.
157. Compute the required sample size.
158. Round the value upward.
159. Display the required number of examples.

Procedure

160. Open the Python/Google Colab environment and prepare the required dataset or input.
161. Import the libraries required for the experiment.
162. Implement the algorithm according to the steps given above.
163. Execute the program and verify the intermediate or final result.
164. Record the obtained output in the space provided.

Program

```
import math

def sample_complexity(H_size, epsilon, delta):
    return math.ceil(
        (1 / epsilon) * (math.log(H_size) + math.log(1 / delta))
    )

for H_size in [10, 100, 1000]:
    m = sample_complexity(H_size, epsilon=0.1, delta=0.05)
    print(f"|H|={H_size}: m >= {m} examples needed")
```

Output

```
|H|=10: m >= 53 examples needed
|H|=100: m >= 77 examples needed
|H|=1000: m >= 100 examples needed
```

Result

The experiment was completed successfully and the result was verified.

Experiment 21 — K-Nearest Neighbour Classifier

Aim

To implement KNN classification on the Iris dataset and evaluate its predictions.

Algorithm

165. Load the Iris dataset.
166. Split the data into training and testing sets.
167. Choose the value of k.
168. Calculate distances from test instances to training instances.
169. Select the k nearest neighbours and use majority voting.
170. Evaluate accuracy and classification metrics.

Procedure

171. Open the Python/Google Colab environment and prepare the required dataset or input.
172. Import the libraries required for the experiment.
173. Implement the algorithm according to the steps given above.
174. Execute the program and verify the intermediate or final result.
175. Record the obtained output in the space provided.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset directly
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Output
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

print("\nPredictions:")
for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print(
        "Actual:", iris.target_names[actual],
```

Output

```
Accuracy: 0.9777777777777777
```

Classification Report:				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	14
versicolor	0.95	1.00	0.97	18
virginica	1.00	0.92	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.98	45
weighted avg	0.98	0.98	0.98	45

Result

The experiment was completed successfully and the result was verified.

Experiment 22 — K-Nearest Neighbour Regression

Aim

To implement KNN regression to predict a continuous target value.

Algorithm

176. Load the regression dataset.
177. Split it into training and testing sets.
178. Choose the number of neighbours.
179. Find the nearest training instances.
180. Average their target values.
181. Evaluate the predictions using RMSE and R^2 .

Procedure

182. Open the Python/Google Colab environment and prepare the required dataset or input.
183. Import the libraries required for the experiment.
184. Implement the algorithm according to the steps given above.
185. Execute the program and verify the intermediate or final result.
186. Record the obtained output in the space provided.

Program

```
# Load dataset
data = fetch_california_housing()
X = data.data
y = data.target

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

# Scale data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# KNN Regression
model = KNeighborsRegressor(n_neighbors=7)
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

# Evaluation
rmse = mean_squared_error(y_test, pred) ** 0.5
r2 = r2_score(y_test, pred)
print("RMSE:", rmse)
print("R2 Score:", r2)

print("\nFirst 10 predictions:")
for actual, predicted in zip(y_test[:10], pred[:10]):
    print(
        "Actual:", round(actual, 2),
        "Predicted:", round(predicted, 2)
    )
```


Output

```
RMSE: 0.6420605967521315
R2 Score: 0.6857172466171257
```

Result

The experiment was completed successfully and the result was verified.

Experiment 23 — Distance-Weighted KNN

Aim

To implement distance-weighted KNN and compare it with uniform-weighted KNN.

Algorithm

187. Load and split the dataset.
188. Create a uniform-weighted KNN classifier.
189. Create a distance-weighted KNN classifier.
190. Train both models.
191. Predict the test samples.
192. Compare their accuracies.

Procedure

193. Open the Python/Google Colab environment and prepare the required dataset or input.
194. Import the libraries required for the experiment.
195. Implement the algorithm according to the steps given above.
196. Execute the program and verify the intermediate or final result.
197. Record the obtained output in the space provided.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

uniform = KNeighborsClassifier(
    n_neighbors=5, weights="uniform"
)

distance = KNeighborsClassifier(
    n_neighbors=5, weights="distance"
)

uniform.fit(X_train, y_train)
distance.fit(X_train, y_train)

print("Uniform-weighted accuracy:",
      accuracy_score(y_test, uniform.predict(X_test)))

print("Distance-weighted accuracy:",
      accuracy_score(y_test, distance.predict(X_test)))
```

Output

```
Uniform-weighted accuracy: 0.9555555555555556
Distance-weighted accuracy: 0.9555555555555556
```

Result

The experiment was completed successfully and the result was verified.

Experiment 24 — Locally Weighted Regression

Aim

To implement Locally Weighted Regression using a Gaussian kernel to fit a non-linear curve.

Algorithm

198. Choose a query point.
199. Calculate Gaussian weights based on distance.
200. Construct the weighted design matrix.
201. Solve the weighted least-squares problem.
202. Predict the query value.
203. Repeat for all query points and plot the fitted curve.

Procedure

204. Open the Python/Google Colab environment and prepare the required dataset or input.
205. Import the libraries required for the experiment.
206. Implement the algorithm according to the steps given above.
207. Execute the program and verify the intermediate or final result.
208. Record the obtained output in the space provided.

Program

```
import numpy as np
import matplotlib.pyplot as plt

def lwr(x_query, X, y, tau=0.5):
    m = len(X)

    weights = np.exp(
        -((X - x_query) ** 2) / (2 * tau ** 2)
    )

    Xb = np.c_[np.ones(m), X]
    W = np.diag(weights)

    theta = np.linalg.pinv(
        Xb.T @ W @ Xb
    ) @ Xb.T @ W @ y

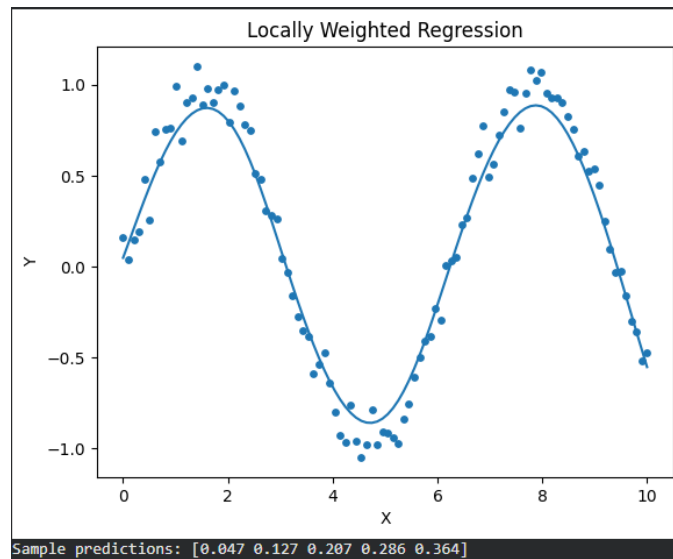
    return np.array([1, x_query]) @ theta

np.random.seed(1)
X = np.linspace(0, 10, 100)
y = np.sin(X) + np.random.normal(0, 0.1, 100)

y_pred = np.array([
    lwr(x, X, y, tau=0.5)
    for x in X
])

plt.scatter(X, y, s=15)
plt.plot(X, y_pred)
plt.title("Locally Weighted Regression")
plt.xlabel("X")
plt.ylabel("Y")
```

Output



Result

The experiment was completed successfully and the result was verified.

Experiment 25 — Radial Basis Function Network

Aim

To implement an RBF network with k-means-selected centres for classification on the Iris dataset.

Algorithm

209. Split the Iris data into training and testing sets.
210. Use k-means to select RBF centres.
211. Calculate Gaussian RBF activations.
212. Transform training and testing data into RBF features.
213. Train a linear classifier on the transformed features.
214. Evaluate classification accuracy.

Procedure

215. Open the Python/Google Colab environment and prepare the required dataset or input.
216. Import the libraries required for the experiment.
217. Implement the algorithm according to the steps given above.
218. Execute the program and verify the intermediate or final result.
219. Record the obtained output in the space provided.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

kmeans = KMeans(
    n_clusters=10,
    random_state=1,
    n_init=10
)

kmeans.fit(X_train)
centres = kmeans.cluster_centers_

sigma = 1.0

def rbf_transform(X, centres, sigma):
    return np.array([
        [
            np.exp(
                -np.sum((x - c) ** 2) /
                (2 * sigma ** 2)
            )
            for c in centres
        ]
        for x in X
    ])

X_train_rbf = rbf_transform(X_train, centres, sigma)
X_test_rbf = rbf_transform(X_test, centres, sigma)
```

Output

Accuracy: 0.9111111111111111

Result

The experiment was completed successfully and the result was verified.

Experiment 26 — Case-Based Reasoning

Aim

To implement a simple Case-Based Reasoning system that retrieves the most similar previous case and reuses its solution.

Algorithm

220. Represent previous cases using features and stored solutions.
221. Define a similarity measure.
222. Compare a new problem with stored cases.
223. Retrieve the most similar case.
224. Reuse or adapt its solution.
225. Retain the new case in the case library.

Procedure

226. Open the Python/Google Colab environment and prepare the required dataset or input.
227. Import the libraries required for the experiment.
228. Implement the algorithm according to the steps given above.

229. Execute the program and verify the intermediate or final result.
230. Record the obtained output in the space provided.

Program

```
def similarity(a, b):  
    return sum(  
        1 for key in a  
        if a[key] == b.get(key)  
    )  
  
def retrieve(new_case):  
    scored = [  
        (similarity(new_case, case["symptoms"]), case)  
        for case in case_library  
    ]  
    return max(scored, key=lambda x: x[0])  
  
new_case = {  
    "fever": True,  
    "cough": True,  
    "fatigue": False  
}  
  
score, best_case = retrieve(new_case)  
  
print("Similarity Score:", score)  
print("Retrieved Diagnosis:",  
      best_case["diagnosis"])  
  
case_library.append({  
    "symptoms": new_case,  
    "diagnosis": best_case["diagnosis"]  
})  
  
print("Case Library Size:",  
      len(case_library))
```

Output

```
Similarity Score: 3  
Retrieved Diagnosis: Flu  
Case Library Size: 4
```

Result

The experiment was completed successfully and the result was verified.

Experiment 27 — NumPy Array Creation and Basic Operations

Aim

To create NumPy arrays using different methods and perform basic mathematical operations.

Algorithm

231. Create arrays using np.array, np.zeros, np.ones, np.arange, and np.linspace.
232. Display array shape, data type, and dimensions.
233. Perform element-wise addition and multiplication.
234. Calculate the dot product.
235. Apply universal functions such as sqrt and exp.
236. Display the results.

Procedure

237. Open the Python/Google Colab environment and prepare the required dataset or input.
238. Import the libraries required for the experiment.

239. Implement the algorithm according to the steps given above.
240. Execute the program and verify the intermediate or final result.
241. Record the obtained output in the space provided.

Program

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.zeros((3, 3))
c = np.ones((2, 4))
d = np.arange(0, 10, 2)
e = np.linspace(0, 1, 5)

print("a:", a)
print("Shape:", a.shape)
print("dtype:", a.dtype)
print("ndim:", a.ndim)

print("Zeros:\n", b)
print("Ones:\n", c)
print("Arange:", d)
print("Linspace:", e)

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print("x+y:", x+y)
print("x*y:", x*y)
print("Dot:", np.dot(x, y))
print("sqrt(x):", np.sqrt(x))
print("exp(x):", np.exp(x))
```

Output

```
a: [1 2 3 4]
Shape: (4,)
dtype: int64
ndim: 1
Zeros:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
Arange: [0 2 4 6 8]
Linspace: [0.    0.25 0.5  0.75 1. ]
x+y: [5 7 9]
x*y: [ 4 10 18]
Dot: 32
sqrt(x): [1.         1.41421356 1.73205081]
exp(x): [ 2.71828183  7.3890561 20.08553692]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 28 — NumPy Indexing and Filtering

Aim

To demonstrate indexing, slicing, and Boolean filtering on multidimensional NumPy arrays.

Algorithm

242. Create and reshape a NumPy array.
243. Access individual elements using indexing.
244. Extract rows or columns using slicing.
245. Filter elements using Boolean conditions.
246. Perform conditional assignment.
247. Display the modified array.

Procedure

248. Open the Python/Google Colab environment and prepare the required dataset or input.
249. Import the libraries required for the experiment.
250. Implement the algorithm according to the steps given above.
251. Execute the program and verify the intermediate or final result.
252. Record the obtained output in the space provided.

Program

```
import numpy as np

a = np.arange(12).reshape(3, 4)

print("Array:\n", a)

print("a[1,2]:", a[1, 2])

print("Column 1:", a[:, 1])

print("Elements > 5:", a[a > 5])

a[a % 2 == 0] = 0

print("After zeroing evens:\n", a)
```

Output

```
Array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
a[1,2]: 6
Column 1: [1 5 9]
Elements > 5: [ 6  7  8  9 10 11]
After zeroing evens:
[[ 0  1  0  3]
 [ 0  5  0  7]
 [ 0  9  0 11]]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 29 — NumPy Statistics and Aggregation

Aim

To compute descriptive statistics using NumPy aggregation functions.

Algorithm

253. Create a two-dimensional array.
254. Calculate the overall mean.
255. Calculate sums along rows and columns.
256. Calculate standard deviation.

257. Find minimum and maximum values.
258. Display the statistical results.

Procedure

259. Open the Python/Google Colab environment and prepare the required dataset or input.
260. Import the libraries required for the experiment.
261. Implement the algorithm according to the steps given above.
262. Execute the program and verify the intermediate or final result.
263. Record the obtained output in the space provided.

Program

```
import numpy as np

a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print("Mean:", a.mean())

print(
    "Sum along axis 0:",
    a.sum(axis=0)
)

print(
    "Sum along axis 1:",
    a.sum(axis=1)
)

print("Std Dev:", a.std())

print(
    "Min per row:",
    a.min(axis=1)
)

print("Max overall:", a.max())
```

Output

```
Mean: 3.5
Sum along axis 0: [5 7 9]
Sum along axis 1: [ 6 15]
Std Dev: 1.707825127659933
Min per row: [1 4]
Max overall: 6
```

Result

The experiment was completed successfully and the result was verified.

Experiment 30 — NumPy Random Module

Aim

To generate random numbers using NumPy's random module for simulation and sampling.

Algorithm

264. Create a seeded random number generator.

265. Generate uniform random values.
266. Generate random integers.
267. Generate normally distributed values.
268. Generate random points for simulation.
269. Estimate π using a Monte Carlo method.

Procedure

270. Open the Python/Google Colab environment and prepare the required dataset or input.
271. Import the libraries required for the experiment.
272. Implement the algorithm according to the steps given above.
273. Execute the program and verify the intermediate or final result.
274. Record the obtained output in the space provided.

Program

```
import numpy as np

rng = np.random.default_rng(seed=42)

print("Uniform random 2x3:")
print(np.round(rng.random((2, 3)), 3))

print("Random integers:")
print(rng.integers(0, 10, size=5))

print("Gaussian samples:")
print(np.round(
    rng.normal(0, 1, size=5), 3
))

n = 100000

points = rng.random((n, 2))

inside = np.sum(
    points[:, 0] ** 2 +
    points[:, 1] ** 2 <= 1
)

print("Estimated pi:",
      4 * inside / n)
```

Output

```
Uniform random 2x3:
[[0.774 0.439 0.859]
 [0.697 0.094 0.976]]
Random integers:
[7 7 7 7 5]
Gaussian samples:
[-0.853  0.879  0.778  0.066  1.127]
Estimated pi: 3.15012
```

Result

The experiment was completed successfully and the result was verified.

Experiment 31 — Saving and Loading Data with NumPy

Aim

To save NumPy arrays in binary and text formats and reload them for later use.

Algorithm

275. Create a NumPy array.
276. Save it using np.save.
277. Save it as text/CSV using np.savetxt.
278. Load the binary file using np.load.
279. Load the text file using np.loadtxt.
280. Display both loaded arrays.

Procedure

281. Open the Python/Google Colab environment and prepare the required dataset or input.
282. Import the libraries required for the experiment.
283. Implement the algorithm according to the steps given above.
284. Execute the program and verify the intermediate or final result.
285. Record the obtained output in the space provided.

Program

```
import numpy as np

a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

np.save("array.npy", a)

np.savetxt(
    "array.csv",
    a,
    delimiter=","
)

loaded_npy = np.load("array.npy")

loaded_csv = np.loadtxt(
    "array.csv",
    delimiter=","
)

print("Loaded from .npy:")
print(loaded_npy)

print("Loaded from .csv:")
print(loaded_csv)
```

Output

```
Loaded from .npy:
[[1 2 3]
 [4 5 6]]
Loaded from .csv:
[[1. 2. 3.]
 [4. 5. 6.] ]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 32 — Pandas DataFrame Creation and Combining

Aim

To create Pandas DataFrames and combine them using concatenation and merging.

Algorithm

286. Create DataFrames from dictionaries.
287. Concatenate DataFrames row-wise.
288. Create another DataFrame with a common key.
289. Merge the DataFrames.
290. Display the concatenated DataFrame.
291. Display the merged DataFrame.

Procedure

292. Open the Python/Google Colab environment and prepare the required dataset or input.
293. Import the libraries required for the experiment.
294. Implement the algorithm according to the steps given above.
295. Execute the program and verify the intermediate or final result.
296. Record the obtained output in the space provided.

Program

```
import pandas as pd
df1 = pd.DataFrame({
    "name": ["A", "B", "C"],
    "marks": [80, 90, 70]
})
df2 = pd.DataFrame({
    "name": ["D"],
    "marks": [85]
})
combined = pd.concat(
    [df1, df2],
    ignore_index=True
)
print("Concatenated:")
print(combined)
df3 = pd.DataFrame({
    "name": ["A", "B", "C"],
    "grade": ["A", "A+", "B"]
})
merged = pd.merge(
    df1,
    df3,
    on="name",
    how="inner"
)
print("Merged:")
print(merged)
```

Output

```
... Concatenated:
   name  marks
0    A     80
1    B     90
2    C     70
3    D     85
Merged:
   name  marks grade
0    A     80    A
1    B     90  A+
2    C     70    B
```

Result

The experiment was completed successfully and the result was verified.

Experiment 33 — Pandas File I/O

Aim

To read data from a CSV file into a DataFrame and write the processed data back to a new CSV file.

Algorithm

297. Read the CSV file using `pd.read_csv`.
298. Inspect the loaded data.
299. Create or modify a column using a condition.
300. Save the processed DataFrame using `to_csv`.
301. Display a preview of the processed data.
302. Verify the output file.

Procedure

303. Open the Python/Google Colab environment and prepare the required dataset or input.
304. Import the libraries required for the experiment.
305. Implement the algorithm according to the steps given above.
306. Execute the program and verify the intermediate or final result.
307. Record the obtained output in the space provided.

Program

```
import pandas as pd

data = pd.DataFrame({
    "name": ["A", "B", "C", "D", "E"],
    "marks": [78, 45, 88, 92, 35],
    "class": ["I", "II", "I", "II", "I"]
})

data.to_csv(
    "students.csv",
    index=False
)

df = pd.read_csv("students.csv")

print("Loaded data:")
print(df)

df["pass"] = df["marks"] >= 50

df.to_csv(
    "students_processed.csv",
    index=False
)

print("Processed data:")
print(df)
```

Output

```
... Loaded data:
   name marks class
0    A    78     I
1    B    45    II
2    C    88     I
3    D    92    II
4    E    35     I
Processed data:
   name marks class pass
0    A    78     I  True
1    B    45    II False
2    C    88     I  True
3    D    92    II  True
4    E    35     I False
```

Result

The experiment was completed successfully and the result was verified.

Experiment 34 — Pandas Grouping and Aggregation

Aim

To group data by a categorical column and compute aggregate statistics.

Algorithm

308. Create or load a DataFrame.
309. Select the categorical grouping column.
310. Apply groupby.
311. Calculate mean values for each group.
312. Calculate multiple aggregates such as mean, maximum, and minimum.
313. Display the grouped results.

Procedure

314. Open the Python/Google Colab environment and prepare the required dataset or input.
315. Import the libraries required for the experiment.
316. Implement the algorithm according to the steps given above.
317. Execute the program and verify the intermediate or final result.
318. Record the obtained output in the space provided.

Program

```
import pandas as pd

df = pd.DataFrame({
    "class": ["I", "II", "I", "II", "I", "II"],
    "marks": [78, 85, 92, 66, 74, 88]
})

print("Mean marks per class:")

print(
    df.groupby("class")["marks"].mean()
)

print("\nMultiple aggregates:")

print(
    df.groupby("class")["marks"].agg(
        ["mean", "max", "min"]
    )
)
```

Output

```
Mean marks per class:
class
I      81.333333
II     79.666667
Name: marks, dtype: float64

Multiple aggregates:
              mean  max  min
class
I      81.333333   92   74
II     79.666667   88   66
```

Result

The experiment was completed successfully and the result was verified.

Experiment 35 — Pandas Filtering and Sorting

Aim

To filter DataFrame rows using conditions and sort the results by one or more columns.

Algorithm

319. Create or load the DataFrame.
320. Define Boolean filtering conditions.
321. Select rows satisfying the conditions.
322. Sort the DataFrame by required columns.
323. Use ascending or descending order as needed.
324. Display filtered and sorted results.

Procedure

325. Open the Python/Google Colab environment and prepare the required dataset or input.
326. Import the libraries required for the experiment.
327. Implement the algorithm according to the steps given above.
328. Execute the program and verify the intermediate or final result.
329. Record the obtained output in the space provided.

Program

```
import pandas as pd

df = pd.DataFrame({
    "name": ["A", "B", "C", "D"],
    "class": ["I", "II", "I", "II"],
    "marks": [78, 85, 92, 66]
})

filtered = df[
    (df["marks"] > 70) &
    (df["class"] == "I")
]

print("Filtered:")
print(filtered)

sorted_df = df.sort_values(
    ["class", "marks"],
    ascending=[True, False]
)

print("\nSorted:")
print(sorted_df)
```

Output

```
Filtered:
  name class marks
0    A     I    78
2    C     I    92

Sorted:
  name class marks
2    C     I    92
0    A     I    78
1    B    II    85
3    D    II    66
```

Result

The experiment was completed successfully and the result was verified.

Experiment 36 — Pandas Plotting

Aim

To visualise DataFrame data using Pandas built-in plotting functions.

Algorithm

330. Create a DataFrame containing numerical data.
331. Plot a histogram of the numerical column.
332. Save the histogram.
333. Create a bar chart.
334. Save the bar chart.
335. Verify the generated plots.

Procedure

336. Open the Python/Google Colab environment and prepare the required dataset or input.
337. Import the libraries required for the experiment.
338. Implement the algorithm according to the steps given above.
339. Execute the program and verify the intermediate or final result.
340. Record the obtained output in the space provided.

Program

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "name": ["A", "B", "C", "D", "E"],
    "marks": [78, 85, 92, 66, 74]
})

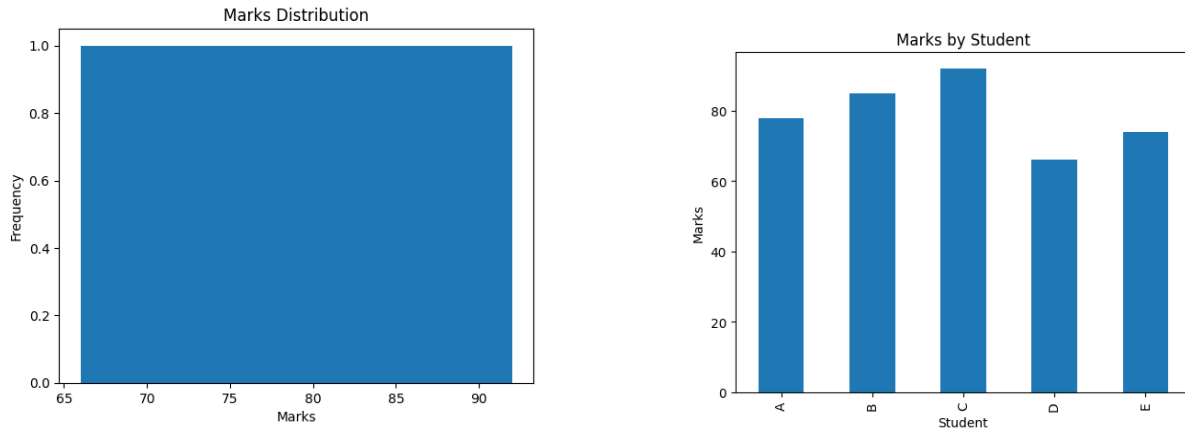
df["marks"].plot(
    kind="hist",
    bins=5,
    title="Marks Distribution"
)

plt.xlabel("Marks")
plt.show()

df.plot(
    x="name",
    y="marks",
    kind="bar",
    legend=False,
    title="Marks by Student"
)

plt.xlabel("Student")
plt.ylabel("Marks")
plt.show()
```

Output



Result

The experiment was completed successfully and the result was verified.

Experiment 37 — Data Cleaning with Pandas

Aim

To identify and handle missing values and duplicate records using Pandas.

Algorithm

341. Create or load a dataset.
342. Detect missing values using `isnull().sum()`.
343. Fill missing values using appropriate replacements.
344. Detect duplicate records.
345. Remove duplicate records.
346. Display the cleaned dataset.

Procedure

347. Open the Python/Google Colab environment and prepare the required dataset or input.
348. Import the libraries required for the experiment.
349. Implement the algorithm according to the steps given above.
350. Execute the program and verify the intermediate or final result.
351. Record the obtained output in the space provided.

Program


```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["A", "B", "B", "C", None],
    "marks": [78, np.nan, 85, 92, 66]
})

print("Missing values:")
print(df.isnull().sum())

df_filled = df.fillna({
    "name": "Unknown",
    "marks": df["marks"].mean()
})

print("\nAfter filling:")
print(df_filled)

df_clean = df_filled.drop_duplicates()

print("\nAfter removing duplicates:")
print(df_clean)
```

Output

```
Missing values:
name      1
marks     1
dtype: int64

After filling:
   name  marks
0     A   78.00
1     B   80.25
2     B   85.00
3     C   92.00
4  Unknown  66.00

After removing duplicates:
   name  marks
0     A   78.00
1     B   80.25
2     B   85.00
3     C   92.00
4  Unknown  66.00
```

Result

The experiment was completed successfully and the result was verified.

Experiment 38 — End-to-End Mini ML Pipeline

Aim

To build a complete machine-learning pipeline from loading and cleaning data to training and prediction.

Algorithm

352. Read the dataset.
353. Handle missing values.
354. Encode categorical labels.
355. Split data into training and testing sets.
356. Scale features and train a KNN classifier.
357. Predict and report accuracy and decoded predictions.

Procedure

358. Open the Python/Google Colab environment and prepare the required dataset or input.
359. Import the libraries required for the experiment.
360. Implement the algorithm according to the steps given above.
361. Execute the program and verify the intermediate or final result.
362. Record the obtained output in the space provided.

Program

```
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=[
        "sepal_length",
        "sepal_width",
        "petal_length",
        "petal_width"
    ]
)

df["species"] = [
    iris.target_names[i]
    for i in iris.target
]

df = df.dropna()

le = LabelEncoder()

df["species_enc"] = le.fit_transform(
    df["species"]
)

X = df.drop(
    columns=["species", "species_enc"]
)
```

Output

```
Pipeline Accuracy: 0.9555555555555556
Sample predictions: [np.str_('setosa') np.str_('versicolor') np.str_('versicolor')
 np.str_('setosa') np.str_('virginica')]
```

Result

The experiment was completed successfully and the result was verified.

Experiment 39 — Model Evaluation Metrics

Aim

To compute and interpret standard classification evaluation metrics for a trained model.

Algorithm

363. Load and split the dataset.
364. Train a classification model.
365. Generate predictions.
366. Compute the confusion matrix.
367. Calculate accuracy, precision, recall, and F1-score.
368. Interpret the evaluation results.

Procedure

369. Open the Python/Google Colab environment and prepare the required dataset or input.
370. Import the libraries required for the experiment.
371. Implement the algorithm according to the steps given above.
372. Execute the program and verify the intermediate or final result.
373. Record the obtained output in the space provided.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=1
)

model = DecisionTreeClassifier(
    random_state=1
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))

print("Accuracy:",
```

Output

```
Confusion Matrix:
[[14  0  0]
 [ 0 17  1]
 [ 0  1 12]]
Accuracy: 0.9555555555555556
Precision: 0.9558404558404558
Recall: 0.9558404558404558
F1-score: 0.9558404558404558
```

Result

The experiment was completed successfully and the result was verified.

Experiment 40 — Complete Machine Learning Pipeline with Visualisation

Aim

To integrate NumPy, Pandas, machine learning models, evaluation, and visualisation into a complete end-to-end pipeline.

Algorithm

374. Load and explore the dataset.
375. Clean and prepare the features.
376. Split the data into training and testing sets.

377. Train two classification models.
378. Evaluate and compare their accuracies.
379. Visualise the model comparison using a bar chart.

Procedure

380. Open the Python/Google Colab environment and prepare the required dataset or input.
381. Import the libraries required for the experiment.
382. Implement the algorithm according to the steps given above.
383. Execute the program and verify the intermediate or final result.
384. Record the obtained output in the space provided.

Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

print("Dataset Summary:")
print(df.describe())

X = df.iloc[:, :-1]
y = df["target"]

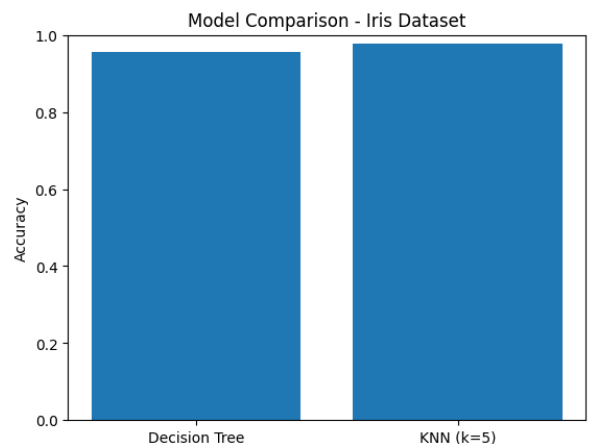
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
```

Output

```
Dataset Summary:
   sepal length (cm)  sepal width (cm)  petal length (cm)  \
count      150.000000      150.000000      150.000000
mean         5.843333         3.057333         3.758000
std          0.828066         0.435866         1.765298
min          4.300000         2.000000         1.000000
25%          5.100000         2.800000         1.600000
50%          5.800000         3.000000         4.350000
75%          6.400000         3.300000         5.100000
max          7.900000         4.400000         6.900000

   petal width (cm)  target
count      150.000000  150.000000
mean         1.199333         1.000000
std          0.762238         0.819232
min          0.100000         0.000000
25%          0.300000         0.000000
50%          1.300000         1.000000
75%          1.800000         2.000000
max          2.500000         2.000000

Model Comparison:
Decision Tree : 0.9556
KNN (k=5) : 0.9778
```



Result

The experiment was completed successfully and the result was verified.